Currying, Partial Application & Composition

Build price pipeline from small functions

Season 6 · Functional · Intermediate

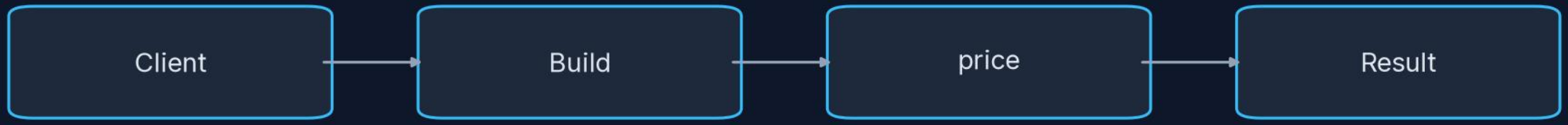
Today's Hook

- Problem: Build price pipeline from small functions
- Naive pain cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- We will refactor to Currying/Composition with a runnable Demo.
- Failure path is locked by ProblemTest.

The Problem

- Scenario: Build price pipeline from small functions
- Constraints: solve «Build price pipeline from small functions» with Currying/Composition — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Build price pipeline from small functions
- Success looks like: 850

Structure



Currying/Composition: Build price pipeline from small functions

Real-World Scenario

- Running example: Build price pipeline from small functions
- Key types: Fns
- Happy-path stdout cue: 850
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```java
package com.buildwithwaqar.patterns.functional.functions.problem;

import java.util.*;

public final class NaivePrice {
  public static int price(int base, int pct, int flat){ return Math.max(0, base*(100-pct)/100 - flat); }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Build price pipeline from small functions
- Without Currying/Composition, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.functional.functions.problem;

import java.util.*;
import java.util.function.*;

final class Fns {
  static Function<Integer, Function<Integer, Integer>> curryAdd(){ return a -> b -> a+b; }
  static <A,B,C> Function<A,C> compose(Function<B,C> g, Function<A,B> f){ return a -> g.apply(f.apply(a)); }
  static Function<Integer,Integer> pricePipeline(int pct, int flat){
    Function<Integer,Integer> percent = c -> c*(100-pct)/100;
    Function<Integer,Integer> flatOff = c -> Math.max(0, c-flat);
    return compose(flatOff, percent);
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.functions.problem;
import java.util.*; import java.util.function.*;
public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Currying/Composition: price pipeline ===");
    System.out.println("--- naive ---"); System.out.println(NaivePrice.price(1000,10,50));
System.out.println("--- curry/compose ---");
System.out.println(Fns.curryAdd().apply(2).apply(3));
System.out.println(Fns.pricePipeline(10,50).apply(1000));
  }
}
```

Walkthrough Output

```
=== Currying/Composition: price pipeline ===
--- naive ---
850
--- curry/compose ---
5
850
```

Step-by-Step Trace

- Match stdout to stages — look for: 850
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp … 50-currying-composition ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
// Java 25 direction for Currying/Composition
// Prefer records + sealed where they shrink the demo.
// Problem: Build price pipeline from small functions
```

Trade-offs

- Use Currying/Composition when «Build price pipeline from small functions» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Currying/Composition in product language.

Related Patterns

- Name the pattern usually confused with Currying/Composition and separate the forces.
- Sketch the roles for «Build price pipeline from small functions» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Build price pipeline from small functions
- Naive pain → Currying/Composition boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.